

```
# =====  
#          STEP 7 – INTERACTIVE POWER BI DASHBOARD PREPARATION  
# =====  
#  
# Goal:  
# Prepare clean, dashboard-ready datasets for Power BI.  
#  
# Recommended Power BI Dashboard Pages:  
#   Page 1 → Executive Overview  
#   Page 2 → Customer Behavior  
#   Page 3 → RFM Customer Segmentation  
#   Page 4 → Churn Analysis  
#   Page 5 → Product & Purchase Patterns  
#  
# =====  
  
# -----  
# STEP 1 – IMPORT LIBRARIES  
# -----  
  
import pandas as pd  
import numpy as np  
from pathlib import Path
```

```
# -----  
# STEP 2 - CREATE OUTPUT FOLDER  
# -----  
  
output_folder = Path("step7_powerbi_data")  
  
output_folder.mkdir(exist_ok=True)  
  
# =====  
# STEP 3 - LOAD DATASETS  
# =====  
  
transactions = pd.read_csv(  
    "ecommerce_cleaned_transactions.csv"  
)  
  
customers = pd.read_csv(  
    "customer_level_features.csv"  
)  
  
rfm = pd.read_csv(  
    "customer_rfm_segments.csv"  
)
```

```
# -----  
# STEP 4 - CONVERT DATE COLUMN  
# -----
```

```
transactions["purchase_date"] = pd.to_datetime(  
    transactions["purchase_date"],  
    errors="coerce"  
)  
|
```

```
# =====  
# STEP 5 - CREATE DATE DIMENSION TABLE  
# =====
```

```
date_min = transactions["purchase_date"].min().normalize()  
date_max = transactions["purchase_date"].max().normalize()
```

```
date_table = pd.DataFrame({  
    "Date": pd.date_range(  
        start=date_min,  
        end=date_max,  
        freq="D"  
    )  
})
```

```
date_table["Year"] = date_table["Date"].dt.year
```

```
date_table["Quarter"] = (
```

```
"Q" +  
date_table["Date"].dt.quarter.astype(str)
```

```
)
```

```
date_table["Month_Number"] = (  
    date_table["Date"].dt.month
```

```
)
```

```
date_table["Month_Name"] = (  
    date_table["Date"].dt.month_name()
```

```
)
```

```
date_table["Month_Year"] = (  
    date_table["Date"].dt.strftime("%Y-%m")
```

```
)
```

```
date_table["Week_Number"] = (  
    date_table["Date"].dt.isocalendar().week.astype(int)
```

```
)
```

```
date_table["Day"] = (  
    date_table["Date"].dt.day
```

```
)
```

```
date_table["Day_Name"] = (  
    date_table["Date"].dt.day_name()
```

```
)
```

```
date_table["Day_Number"] = (  
    date_table["Date"].dt.dayofweek + 1  
)
```

```
date_table["Is_Weekend"] = (  
    date_table["Day_Number"] >= 6  
)
```

```
date_table["Year_Month_Sort"] = (  
    date_table["Year"] * 100 +  
    date_table["Month_Number"]  
)
```

```
# -----  
# STEP 6 - DISPLAY DATE TABLE  
# -----
```

```
print("\n===== DATE DIMENSION =====\n")
```

```
display(date_table.head())
```

```
# =====  
# STEP 7 - CREATE PRODUCT DIMENSION  
# =====  
  
product_columns = [  
    "product_category"  
]  
  
product_dimension = (  
    transactions[product_columns]  
    .drop_duplicates()  
    .sort_values("product_category")  
    .reset_index(drop=True)  
)  
  
product_dimension["Product_Category_ID"] = (  
    product_dimension.index + 1  
)  
  
product_dimension = product_dimension[  
    [  
        "Product_Category_ID",  
        "product_category"  
    ]  
]
```

```

# -----
# STEP 8 - DISPLAY PRODUCT DIMENSION
# -----

print("\n===== PRODUCT DIMENSION =====\n")

display(product_dimension)


# =====
# STEP 9 - CREATE PAYMENT DIMENSION
# =====

payment_dimension = (
    transactions[["payment_method"]]
    .drop_duplicates()
    .sort_values("payment_method")
    .reset_index(drop=True)
)

payment_dimension["Payment_Method_ID"] = (
    payment_dimension.index + 1
)

payment_dimension = payment_dimension[
    [
        "Payment_Method_ID",
        "payment_method"
    ]

```

```

    ]
]

# -----
# STEP 10 - DISPLAY PAYMENT DIMENSION
# -----

print("\n===== PAYMENT DIMENSION =====\n")

display(payment_dimension)

# =====
# STEP 11 - CREATE CUSTOMER DIMENSION
# =====

customer_dimension_columns = [
    "customer_id"
]

optional_customer_columns = [
    "customer_name",
    "customer_age",
    "gender"
]

for col in optional_customer_columns:

```

```
if col in customers.columns:
    customer_dimension_columns.append(col)
```

```
customer_dimension = (
    customers[customer_dimension_columns]
    .drop_duplicates("customer_id")
    .copy()
)
```

```
customer_dimension = customer_dimension.sort_values(
    "customer_id"
).reset_index(drop=True)
```

```
# -----
# STEP 12 – ADD CUSTOMER SEGMENT INFORMATION
# -----
```

```
if "Customer_Segment" in rfm.columns:
```

```
    segment_columns = [
        "customer_id",
        "Customer_Segment"
    ]
```

```
customer_segment_data = (
    rfm[segment_columns]
    .drop_duplicates("customer_id")
)
```

```
)

customer_dimension = customer_dimension.merge(
    customer_segment_data,
    on="customer_id",
    how="left"
)

# -----
# STEP 13 - ADD RFM SCORES
# -----

rfm_score_columns = [
    "customer_id",
    "R_Score",
    "F_Score",
    "M_Score",
    "RFM_Score"
]

available_rfm_score_columns = [
    col for col in rfm_score_columns
    if col in rfm.columns
]

if "customer_id" in available_rfm_score_columns:
```

```
customer_rfm_scores = (  
    rfm[available_rfm_score_columns]  
    .drop_duplicates("customer_id")  
)  
  
customer_dimension = customer_dimension.merge(  
    customer_rfm_scores,  
    on="customer_id",  
    how="left"  
)
```

```
# =====  
# STEP 14 - CREATE CUSTOMER AGE GROUP  
# =====
```

```
if "customer_age" in customer_dimension.columns:
```

```
    customer_dimension["Age_Group"] = pd.cut(  
        customer_dimension["customer_age"],  
        bins=[  
            0,  
            18,  
            25,  
            35,  
            45,  
            55,  
            65,|
```

```

100
    ],
    labels=[
        "Under 18",
        "18-25",
        "26-35",
        "36-45",
        "46-55",
        "56-65",
        "65+"
    ],
    include_lowest=True
)

```

```

# =====
# STEP 15 – CREATE CUSTOMER VALUE CATEGORY
# =====

```

```

if "monetary" in rfm.columns:

    customer_value = rfm[
        [
            "customer_id",
            "monetary"
        ]
    ].copy()

    customer_value["Customer_Value_Category"] = pd.qcut(

```

```

customer_value["monetary"].rank(method="first"),
q=4,
labels=[
    "Low Value",
    "Medium Value",
    "High Value",
    "Very High Value"
]
)

```

```

customer_dimension = customer_dimension.merge(
    customer_value[
        [
            "customer_id",
            "Customer_Value_Category"
        ]
    ],
    on="customer_id",
    how="left"
)

```

```

# =====
# STEP 16 – CREATE TRANSACTION FACT TABLE
# =====

```

```

fact_sales = transactions.copy()
|

```

```
# Create a unique transaction ID
```

```
fact_sales.insert(  
    0,  
    "Transaction_ID",  
    range(1, len(fact_sales) + 1)  
)
```

```
# -----
```

```
# STEP 17 - CREATE DATE KEY
```

```
# -----
```

```
fact_sales["Date"] = (  
    fact_sales["purchase_date"].dt.normalize()  
)
```

```
# -----
```

```
# STEP 18 - CREATE YEAR / MONTH / QUARTER
```

```
# -----
```

```
fact_sales["Year"] = (  
    fact_sales["purchase_date"].dt.year  
)
```

```
fact_sales["Quarter"] = (  
    "Q" +  
    fact_sales["purchase_date"]
```

```
.dt.quarter  
.astype(str)
```

```
)
```

```
fact_sales["Month_Number"] = (  
    fact_sales["purchase_date"].dt.month
```

```
)
```

```
fact_sales["Month_Name"] = (  
    fact_sales["purchase_date"].dt.month_name()
```

```
)
```

```
fact_sales["Month_Year"] = (  
    fact_sales["purchase_date"]  
    .dt.strftime("%Y-%m")
```

```
)
```

```
fact_sales["Day_Name"] = (  
    fact_sales["purchase_date"]  
    .dt.day_name()
```

```
)
```

```
fact_sales["Purchase_Hour"] = (  
    fact_sales["purchase_date"].dt.hour
```

```
)
```

```
# =====  
# STEP 19 - ADD PURCHASE TIME PERIOD  
# =====
```

```
def assign_time_period(hour):
```

```
    if 5 <= hour < 12:
```

```
        return "Morning"
```

```
    elif 12 <= hour < 17:
```

```
        return "Afternoon"
```

```
    elif 17 <= hour < 21:
```

```
        return "Evening"
```

```
    else:
```

```
        return "Night"
```

```
fact_sales["Time_Period"] = (  
    fact_sales["Purchase_Hour"]  
    .apply(assign_time_period)  
)
```

```

# =====
# STEP 20 - ADD WEEKEND FLAG
# =====

fact_sales["Is_Weekend"] = (
    fact_sales["purchase_date"].dt.dayofweek >= 5
)

# =====
# STEP 21 - CREATE PRICE BAND
# =====

if "product_price" in fact_sales.columns:

    fact_sales["Price_Band"] = pd.cut(
        fact_sales["product_price"],
        bins=[
            -np.inf,
            50,
            100,
            250,
            500,
            np.inf
        ],
        labels=[
            "Below 50",
            "50-100",
            "101-250",

```

```
        "251-500",  
        "500+"  
    ]  
)
```

```
# =====  
# STEP 22 - CREATE QUANTITY BAND  
# =====
```

```
if "quantity" in fact_sales.columns:
```

```
    fact_sales["Quantity_Band"] = pd.cut(  
        fact_sales["quantity"],  
        bins=[  
            0,  
            1,  
            2,  
            3,  
            5,  
            np.inf  
        ],  
        labels=[  
            "1",  
            "2",  
            "3",  
            "4-5",  
            "6+"  
        ]  
    )
```

```
)  
  
# =====  
# STEP 23 - ADD PRODUCT CATEGORY ID  
# =====
```

```
fact_sales = fact_sales.merge(  
    product_dimension,  
    on="product_category",  
    how="left"  
)
```

```
# =====  
# STEP 24 - ADD PAYMENT METHOD ID  
# =====
```

```
fact_sales = fact_sales.merge(  
    payment_dimension,  
    on="payment_method",  
    how="left"  
)
```

```

# =====
# STEP 25 – CREATE REVENUE PER TRANSACTION
# =====

fact_sales["Revenue"] = (
    fact_sales["total_purchase_amount"]
)

# =====
# STEP 26 – CREATE RETURN FLAG
# =====

if "returns" in fact_sales.columns:

    fact_sales["Return_Flag"] = (
        fact_sales["returns"] > 0
    ).astype(int)

else:

    fact_sales["Return_Flag"] = 0

# =====
# STEP 27 – CREATE CUSTOMER KPI TABLE
# =====

customer_kpis = (

```

```
fact_sales
.groupby("customer_id")
.agg(
    Total_Spend=(
        "total_purchase_amount",
        "sum"
    ),
    Purchase_Frequency=(
        "Transaction_ID",
        "count"
    ),
    Total_Quantity=(
        "quantity",
        "sum"
    ),
    Average_Order_Value=(
        "total_purchase_amount",
        "mean"
    ),
    Average_Product_Price=(
        "product_price",
        "mean"
    ),
    Total_Returns=(
        "Return_Flag",
        "sum"
    ),
    First_Purchase_Date=(
        "purchase_date",|
```

```

        "min"
    ),
    Last_Purchase_Date=(
        "purchase_date",
        "max"
    )
)
.reset_index()
)

# =====
# STEP 28 – CALCULATE CUSTOMER RECENCY
# =====

analysis_date = (
    fact_sales["purchase_date"].max()
    .normalize()
)

customer_kpis["Recency"] = (
    analysis_date -
    customer_kpis["Last_Purchase_Date"]
).dt.days

```

```
# STEP 29 - CALCULATE RETURN RATE
```

```
# =====
```

```
customer_kpis["Return_Rate"] = np.where(  
    customer_kpis["Purchase_Frequency"] > 0,  
    (  
        customer_kpis["Total>Returns"] /  
        customer_kpis["Purchase_Frequency"]  
    ) * 100,  
    0  
)
```

```
# =====
```

```
# STEP 30 - MERGE CHURN INFORMATION
```

```
# =====
```

```
if {  
    "customer_id",  
    "churn"  
}.issubset(rfm.columns):
```

```
    churn_data = (  
        rfm[  
            [  
                "customer_id",  
                "churn"  
            ]  
        ]
```

```
.drop_duplicates("customer_id")  
  
)  
  
customer_kpis = customer_kpis.merge(  
    churn_data,  
    on="customer_id",  
    how="left"  
)
```

```
# =====  
# STEP 31 - CREATE CHURN STATUS  
# =====
```

```
if "churn" in customer_kpis.columns:  
  
    customer_kpis["Churn_Status"] = np.where(  
        customer_kpis["churn"] == 1,  
        "Churned",  
        "Active"  
    )
```

```
# =====  
# STEP 32 - CREATE CHURN RISK  
# =====  
  
if {  
    "R_Score",  
    "F_Score",  
    "churn"  
}.issubset(rfm.columns):  
  
    risk_data = rfm[  
        [  
            "customer_id",  
            "R_Score",  
            "F_Score"  
        ]  
    ].drop_duplicates("customer_id")  
  
    customer_kpis = customer_kpis.merge(  
        risk_data,  
        on="customer_id",  
        how="left"  
    )  
  
    def calculate_risk(row):  
  
        if row["churn"] == 1:  
            return "Churned"
```

```

elif (
    row["R_Score"] <= 2 and
    row["F_Score"] <= 2
):
    return "High Risk"

elif row["R_Score"] <= 2:
    return "Medium Risk"

elif row["R_Score"] >= 4:
    return "Low Risk"

else:
    return "Moderate Risk"

customer_kpis["Churn_Risk"] = (
    customer_kpis.apply(
        calculate_risk,
        axis=1
    )
)

```

```

# =====
# STEP 33 - CREATE CUSTOMER KPI CATEGORIES
# =====

```

```

customer_kpis["Spend_Category"] = pd.qcut(

```

```

customer_kpis["Total_Spend"].rank(method="first"),
q=4,
labels=[
    "Low",
    "Medium",
    "High",
    "Very High"
]
)

customer_kpis["Frequency_Category"] = pd.qcut(
    customer_kpis["Purchase_Frequency"].rank(method="first"),
    q=4,
    labels=[
        "Low",
        "Medium",
        "High",
        "Very High"
    ]
)

customer_kpis["Recency_Category"] = pd.cut(
    customer_kpis["Recency"],
    bins=[
        -1,
        30,
        90,
        180,|

```

```

        365,
        np.inf
    ],
    labels=[
        "0-30 Days",
        "31-90 Days",
        "91-180 Days",
        "181-365 Days",
        "365+ Days"
    ]
)

```

```

# =====
# STEP 34 – CREATE MONTHLY SALES FACT TABLE
# =====

```

```

monthly_sales = (
    fact_sales
    .groupby(
        [
            "Year",
            "Month_Number",
            "Month_Name",
            "Month_Year"
        ],
        as_index=False
    )
    .agg([

```

```
Revenue=(  
    "total_purchase_amount",  
    "sum"  
)  
Transactions=(  
    "Transaction_ID",  
    "count"  
)  
Quantity=(  
    "quantity",  
    "sum"  
)  
Customers=(  
    "customer_id",  
    "nunique"  
)  
)  
)  
  
monthly_sales["Average_Order_Value"] = (  
    monthly_sales["Revenue"] /  
    monthly_sales["Transactions"]  
)
```

```
# =====  
# STEP 35 - CREATE CATEGORY PERFORMANCE TABLE  
# =====
```

```
category_performance = (  
    fact_sales  
    .groupby("product_category", as_index=False)  
    .agg(  
        Revenue=(  
            "total_purchase_amount",  
            "sum"  
        ),  
        Transactions=(  
            "Transaction_ID",  
            "count"  
        ),  
        Customers=(  
            "customer_id",  
            "nunique"  
        ),  
        Quantity=(  
            "quantity",  
            "sum"  
        ),  
        Average_Order_Value=(  
            "total_purchase_amount",  
            "mean"  
        ),  
        Return_Count=(
```

```
"Return_Flag",
```

```
"sum"
```

```
)
```

```
)
```

```
)
```

```
category_performance["Return_Rate"] = np.where(
```

```
    category_performance["Transactions"] > 0,
```

```
    (
```

```
        category_performance["Return_Count"] /
```

```
        category_performance["Transactions"]
```

```
    ) * 100,
```

```
    0
```

```
)
```

```
# =====
```

```
# STEP 36 – CREATE PAYMENT PERFORMANCE TABLE
```

```
# =====
```

```
payment_performance = (
```

```
    fact_sales
```

```
    .groupby("payment_method", as_index=False)
```

```
    .agg(
```

```
        Revenue=(
```

```
            "total_purchase_amount",
```

```
            "sum"
```

```
        ),
```

```
        Transactions=(
```

```

        "Transaction_ID",
        "count"
    ),
    Customers=(
        "customer_id",
        "nunique"
    ),
    Average_Order_Value=(
        "total_purchase_amount",
        "mean"
    )
)
)

```

```

# =====
# STEP 37 – CREATE DAY OF WEEK TABLE
# =====

```

```

day_order = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
]

```

```

day_performance = (
    fact_sales
    .groupby("Day_Name", as_index=False)
    .agg(
        Revenue=(
            "total_purchase_amount",
            "sum"
        ),
        Transactions=(
            "Transaction_ID",
            "count"
        ),
        Customers=(
            "customer_id",
            "nunique"
        )
    )
)

day_performance["Day_Name"] = pd.Categorical(
    day_performance["Day_Name"],
    categories=day_order,
    ordered=True
)

day_performance = (
    day_performance
    .sort_values("Day_Name")

```

```
.reset_index(drop=True)
)

# =====
# STEP 38 - CREATE HOURLY PERFORMANCE TABLE
# =====

hour_performance = (
    fact_sales
    .groupby("Purchase_Hour", as_index=False)
    .agg(
        Revenue=(
            "total_purchase_amount",
            "sum"
        ),
        Transactions=(
            "Transaction_ID",
            "count"
        ),
        Customers=(
            "customer_id",
            "nunique"
        )
    )
    .sort_values("Purchase_Hour")
)
```

```
# =====  
# STEP 39 - CREATE TIME PERIOD PERFORMANCE  
# =====
```

```
time_period_performance = (  
    fact_sales  
    .groupby("Time_Period", as_index=False)  
    .agg(  
        Revenue=(  
            "total_purchase_amount",  
            "sum"  
        ),  
        Transactions=(  
            "Transaction_ID",  
            "count"  
        ),  
        Customers=(  
            "customer_id",  
            "nunique"  
        )  
    )  
)
```

```
# =====  
# STEP 40 – CREATE CHURN SUMMARY  
# =====  
  
if "churn" in customer_kpis.columns:  
  
    churn_summary = (  
        customer_kpis  
        .groupby("Churn_Status", as_index=False)  
        .agg(  
            Customers=(  
                "customer_id",  
                "nunique"  
            ),  
            Total_Spend=(  
                "Total_Spend",  
                "sum"  
            ),  
            Average_Spend=(  
                "Total_Spend",  
                "mean"  
            ),  
            Average_Frequency=(  
                "Purchase_Frequency",  
                "mean"  
            ),  
            Average_Recency=(  
                "Recency",  
                "mean"  
            )  
        )  
    )
```

```
)  
    )  
)
```

else:

```
churn_summary = pd.DataFrame()
```

```
# =====  
# STEP 41 - CREATE RFM SEGMENT SUMMARY  
# =====
```

```
if "Customer_Segment" in rfm.columns:
```

```
    rfm_segment_summary = (  
        rfm  
        .groupby(  
            "Customer_Segment",  
            as_index=False  
        )  
        .agg(  
            Customers=(  
                "customer_id",  
                "nunique"  
            ),  
            Revenue=(  
                "monetary",
```

```

        "sum"
    ),
    Average_Spend=(
        "monetary",
        "mean"
    ),
    Average_Frequency=(
        "frequency",
        "mean"
    ),
    Average_Recency=(
        "recency",
        "mean"
    ),
    Average_RFM_Score=(
        "RFM_Score",
        "mean"
    )
)
)

```

else:

```

rfm_segment_summary = pd.DataFrame()

```

```
# =====  
# STEP 42 - CREATE DASHBOARD KPI TABLE  
# =====
```

```
dashboard_kpis = pd.DataFrame({  
    "KPI": [  
        "Total Customers",  
        "Total Transactions",  
        "Total Revenue",  
        "Average Order Value",  
        "Total Quantity",  
        "Total Returns",  
        "Churned Customers",  
        "Churn Rate"  
    ],  
    "Value": [  
        fact_sales["customer_id"].nunique(),  
        len(fact_sales),  
        fact_sales["total_purchase_amount"].sum(),  
        fact_sales["total_purchase_amount"].mean(),  
        fact_sales["quantity"].sum(),  
        fact_sales["Return_Flag"].sum(),  
        (  
            customer_kpis["churn"].sum()  
            if "churn" in customer_kpis.columns  
            else np.nan  
        ),  
    ]  
})
```

```

        customer_kpis["churn"].mean() * 100
    if "churn" in customer_kpis.columns
    else np.nan
)
]
})

# =====
# STEP 43 - CREATE HIGH-VALUE AT-RISK TABLE
# =====

if {
    "R_Score",
    "M_Score",
    "churn"
}.issubset(rfm.columns):

    high_value_at_risk = rfm[
        (
            (rfm["R_Score"] <= 2) &
            (rfm["M_Score"] >= 4) &
            (rfm["churn"] == 0)
        )
    ].copy()

else:

```

```
high_value_at_risk = pd.DataFrame()
```

```
# =====  
# STEP 44 – CREATE CUSTOMER SEGMENT × CHURN TABLE  
# =====
```

```
if {  
    "Customer_Segment",  
    "churn"  
}.issubset(rfm.columns):  
  
    segment_churn = (  
        rfm  
        .groupby(  
            "Customer_Segment",  
            as_index=False  
        )  
        .agg(  
            Customers=(  
                "customer_id",  
                "nunique"  
            ),  
            Churned_Customers=(  
                "churn",  
                "sum"  
            )  
        )  
    )
```

```

)

segment_churn["Churn_Rate"] = np.where(
    segment_churn["Customers"] > 0,
    (
        segment_churn["Churned_Customers"] /
        segment_churn["Customers"]
    ) * 100,
    0
)

```

else:

```

segment_churn = pd.DataFrame()

```

```

# =====
# STEP 45 - SAVE DATE DIMENSION
# =====

```

```

date_table.to_csv(
    output_folder / "dim_date.csv",
    index=False
)

```

```
# =====  
# STEP 46 - SAVE CUSTOMER DIMENSION  
# =====  
  
customer_dimension.to_csv(  
    output_folder / "dim_customer.csv",  
    index=False  
)  
  
# =====  
# STEP 47 - SAVE PRODUCT DIMENSION  
# =====  
  
product_dimension.to_csv(  
    output_folder / "dim_product.csv",  
    index=False  
)  
  
# =====  
# STEP 48 - SAVE PAYMENT DIMENSION  
# =====  
  
payment_dimension.to_csv(  
    output_folder / "dim_payment.csv",  
    index=False  
)
```

```
# =====  
# STEP 49 – SAVE SALES FACT TABLE  
# =====  
  
fact_sales.to_csv(  
    output_folder / "fact_sales.csv",  
    index=False  
)  
  
# =====  
# STEP 50 – SAVE CUSTOMER KPI TABLE  
# =====  
  
customer_kpis.to_csv(  
    output_folder / "customer_kpis.csv",  
    index=False  
)  
  
# =====  
# STEP 51 – SAVE MONTHLY SALES  
# =====  
  
monthly_sales.to_csv(  
    output_folder / "monthly_sales.csv",  
    index=False  
)
```

```
# =====  
# STEP 52 – SAVE CATEGORY PERFORMANCE  
# =====  
  
category_performance.to_csv(  
    output_folder / "category_performance.csv",  
    index=False  
)  
  
# =====  
# STEP 53 – SAVE PAYMENT PERFORMANCE  
# =====  
  
payment_performance.to_csv(  
    output_folder / "payment_performance.csv",  
    index=False  
)  
  
# =====  
# STEP 54 – SAVE DAY PERFORMANCE  
# =====  
  
day_performance.to_csv(  
    output_folder / "day_performance.csv",  
    index=False  
)
```

```
# =====  
# STEP 55 – SAVE HOURLY PERFORMANCE  
# =====  
  
hour_performance.to_csv(  
    output_folder / "hour_performance.csv",  
    index=False  
)  
  
# =====  
# STEP 56 – SAVE TIME PERIOD PERFORMANCE  
# =====  
  
time_period_performance.to_csv(  
    output_folder / "time_period_performance.csv",  
    index=False  
)  
  
# =====  
# STEP 57 – SAVE CHURN SUMMARY  
# =====  
  
if not churn_summary.empty:  
  
    churn_summary.to_csv(  
        output_folder / "churn_summary.csv",
```

```
index=False
```

```
)
```

```
# =====
```

```
# STEP 58 – SAVE RFM SEGMENT SUMMARY
```

```
# =====
```

```
if not rfm_segment_summary.empty:
```

```
    rfm_segment_summary.to_csv(  
        output_folder / "rfm_segment_summary.csv",  
        index=False
```

```
)
```

```
# =====
```

```
# STEP 59 – SAVE DASHBOARD KPIS
```

```
# =====
```

```
dashboard_kpis.to_csv(  
    output_folder / "dashboard_kpis.csv",  
    index=False
```

```
)
```

```

# =====
# STEP 60 – SAVE HIGH-VALUE AT-RISK CUSTOMERS
# =====

if not high_value_at_risk.empty:

    high_value_at_risk.to_csv(
        output_folder / "high_value_at_risk.csv",
        index=False
    )

# =====
# STEP 61 – SAVE SEGMENT CHURN
# =====

if not segment_churn.empty:

    segment_churn.to_csv(
        output_folder / "segment_churn.csv",
        index=False
    )

# =====
# STEP 62 – CREATE POWER BI DATA DICTIONARY
# =====

data_dictionary = pd.DataFrame({

```

```
"Table": [  
    "fact_sales",  
    "dim_customer",  
    "dim_product",  
    "dim_payment",  
    "dim_date",  
    "customer_kpis"  
],  
"Purpose": [  
    "Transaction-level sales data",  
    "Customer master and segmentation data",  
    "Product category dimension",  
    "Payment method dimension",  
    "Date dimension for time analysis",  
    "Customer-level KPIs and churn metrics"  
]  
})
```

```
data_dictionary.to_csv(  
    output_folder / "data_dictionary.csv",  
    index=False  
)
```

```
# =====  
# STEP 63 – POWER BI MODEL RELATIONSHIPS  
# =====
```

```
relationships = pd.DataFrame({  
    "From_Table": [  
        "fact_sales",  
        "fact_sales",  
        "fact_sales",  
        "fact_sales"  
    ],  
    "From_Column": [  
        "customer_id",  
        "Product_Category_ID",  
        "Payment_Method_ID",  
        "Date"  
    ],  
    "To_Table": [  
        "dim_customer",  
        "dim_product",  
        "dim_payment",  
        "dim_date"  
    ],  
    "To_Column": [  
        "customer_id",  
        "Product_Category_ID",  
        "Payment_Method_ID",  
        "Date"  
    ]  
})
```

```

    ],
    "Cardinality": [
        "Many-to-One",
        "Many-to-One",
        "Many-to-One",
        "Many-to-One"
    ]
})

```

```

relationships.to_csv(
    output_folder / "powerbi_relationships.csv",
    index=False
)

```

```

# =====
# STEP 64 – POWER BI DAX MEASURES
# =====

```

```

dax_measures = """
Total Revenue =
SUM(fact_sales[total_purchase_amount])

```

```

Total Transactions =
COUNTROWS(fact_sales)

```

```

Total Customers =
DISTINCTCOUNT(fact_sales[customer_id])

```

Total Quantity =
SUM(fact_sales[quantity])

Average Order Value =
DIVIDE(
 [Total Revenue],
 [Total Transactions]
)

Average Quantity per Transaction =
DIVIDE(
 [Total Quantity],
 [Total Transactions]
)

Total Returns =
SUM(fact_sales[Return_Flag])

Return Rate =
DIVIDE(
 [Total Returns],
 [Total Transactions]
) * 100

Churned Customers =
CALCULATE(
 DISTINCTCOUNT(customer_kpis[customer_id]),

```
customer_kpis[churn] = 1
)
```

```
Churn Rate =
DIVIDE(
    [Churned Customers],
    [Total Customers]
) * 100
```

```
Active Customers =
CALCULATE(
    DISTINCTCOUNT(customer_kpis[customer_id]),
    customer_kpis[churn] = 0
)
```

```
Average Customer Spend =
AVERAGE(customer_kpis[Total_Spend])
```

```
Average Customer Frequency =
AVERAGE(customer_kpis[Purchase_Frequency])
```

```
Average Customer Recency =
AVERAGE(customer_kpis[Recency])
```

```
High Value At Risk =
CALCULATE(
    DISTINCTCOUNT(customer_kpis[customer_id]),
    customer_kpis[Churn_Risk] = "High Risk"
```

```

)

Revenue from Churned Customers =
CALCULATE(
    SUM(customer_kpis[Total_Spend]),
    customer_kpis[churn] = 1
)

Revenue from Active Customers =
CALCULATE(
    SUM(customer_kpis[Total_Spend]),
    customer_kpis[churn] = 0
)
"""

with open(
    output_folder / "recommended_dax_measures.txt",
    "w",
    encoding="utf-8"
) as file:

    file.write(dax_measures)

```

```
# =====  
# STEP 65 – DISPLAY DASHBOARD KPIS  
# =====  
  
print("\n=====")  
print(" STEP 7 – POWER BI DATA PREPARATION COMPLETED")  
print("=====\n")  
  
display(dashboard_kpis)  
  
# =====  
# STEP 66 – DISPLAY POWER BI MODEL  
# =====  
  
print("\n===== POWER BI MODEL =====\n")  
  
display(relationships)  
  
# =====  
# STEP 67 – DISPLAY GENERATED FILES  
# =====  
  
print("\n===== GENERATED FILES =====\n")  
  
for file in sorted(output_folder.iterdir()):
```

```
print("✓", file.name)
```

```
# =====  
# STEP 68 - FINAL MESSAGE  
# =====
```

```
print("\n=====")  
print("POWER BI DATA MODEL IS READY")  
print("=====")
```

```
print("\nRecommended Power BI Pages:")
```

```
print("1. Executive Overview")  
print("2. Customer Behavior")  
print("3. RFM Segmentation")  
print("4. Churn Analysis")  
print("5. Product & Purchase Patterns")
```

```
print("\nRecommended relationships:")  
print("fact_sales → dim_customer")  
print("fact_sales → dim_product")  
print("fact_sales → dim_payment")  
print("fact_sales → dim_date")
```

```
print("\nStep 7 completed successfully.")
```

===== DATE DIMENSION =====

	Date	Year	Quarter	Month_Number	Month_Name	Month_Year	Week_Number	Day	Day_Name	Day_Number	Is_Weekend	Year_Month_Sort
0	2020-01-01	2020	Q1	1	January	2020-01	1	1	Wednesday	3	False	202001
1	2020-01-02	2020	Q1	1	January	2020-01	1	2	Thursday	4	False	202001
2	2020-01-03	2020	Q1	1	January	2020-01	1	3	Friday	5	False	202001
3	2020-01-04	2020	Q1	1	January	2020-01	1	4	Saturday	6	True	202001
4	2020-01-05	2020	Q1	1	January	2020-01	1	5	Sunday	7	True	202001

===== PRODUCT DIMENSION =====

	Product_Category_ID	product_category
0	1	Books
1	2	Clothing
2	3	Electronics
3	4	Home

===== PAYMENT DIMENSION =====

	Payment_Method_ID	payment_method
0	1	Cash
1	2	Credit Card
2	3	Crypto
3	4	PayPal

=====

STEP 7 – POWER BI DATA PREPARATION COMPLETED

=====

	KPI	Value
0	Total Customers	4.967300e+04
1	Total Transactions	2.500000e+05
2	Total Revenue	6.813427e+08
3	Average Order Value	2.725371e+03
4	Total Quantity	7.497240e+05

5	Total Returns	1.007690e+05
6	Churned Customers	9.942000e+03
7	Churn Rate	2.001490e+01

===== POWER BI MODEL =====

	From_Table	From_Column	To_Table	To_Column	Cardinality
0	fact_sales	customer_id	dim_customer	customer_id	Many-to-One
1	fact_sales	Product_Category_ID	dim_product	Product_Category_ID	Many-to-One
2	fact_sales	Payment_Method_ID	dim_payment	Payment_Method_ID	Many-to-One
3	fact_sales	Date	dim_date	Date	Many-to-One

===== GENERATED FILES =====

- ✓ category_performance.csv
- ✓ churn_summary.csv
- ✓ customer_kpis.csv
- ✓ dashboard_kpis.csv
- ✓ data_dictionary.csv
- ✓ day_performance.csv
- ✓ dim_customer.csv
- ✓ dim_date.csv
- ✓ dim_payment.csv

- ✓ dim_product.csv
- ✓ fact_sales.csv
- ✓ high_value_at_risk.csv
- ✓ hour_performance.csv
- ✓ monthly_sales.csv
- ✓ payment_performance.csv
- ✓ powerbi_relationships.csv
- ✓ recommended_dax_measures.txt
- ✓ rfm_segment_summary.csv
- ✓ segment_churn.csv
- ✓ time_period_performance.csv

=====

POWER BI DATA MODEL IS READY

=====

Recommended Power BI Pages:

1. Executive Overview
2. Customer Behavior
3. RFM Segmentation
4. Churn Analysis
5. Product & Purchase Patterns

Recommended relationships:

fact_sales → dim_customer
fact_sales → dim_product
fact_sales → dim_payment
fact_sales → dim_date

Step 7 completed successfully.